

IntrecciAmi: Detailed Summary from Fine-Tuning to Evaluation

Expanded version combining fine-tuning setup, architecture details, evaluation metrics, value ranges, and MLLM judging.

Prepared from project files under Captioning/, including final_report.md, phase6 instructions, training summaries, automated metrics reports, and mllm judge reports.

Table of Contents

- 1. Project story and timeline
- 2. Why metadata and normalization mattered before training
- 3. LoRA concept and where it was attached
- 4. Fine-tuning setup for FLUX, SDXL, and Z-Image
- 5. Internal architecture explanation of SDXL
- 6. Internal architecture explanation of FLUX.1-dev
- 7. Internal architecture explanation of Z-Image
- 8. Phase 6 generalization prompt set
- 9. Automated evaluation pipeline: CLIPScore, LPIPS, CLIP-IQA
- 10. Range and interpretation of metric values
- 11. MLLM-as-a-judge pipeline and 0 to 5 scoring
- 12. Final model-by-model interpretation

1. Project story and timeline

The IntrecciAmi project asked a difficult question: can we transfer the language and structure of artisan weaving into image generation models in a controlled way? The project did not start from a clean text-to-image dataset. It started from workshop photographs, Label Studio annotations, metadata extraction, and caption generation. Only after that preparation could fine-tuning begin.

From the fine-tuning stage onward, the workflow can be summarized like this:

- Prepare model-specific datasets built from the normalized metadata and generated captions.
- Fine-tune three backbones with LoRA: FLUX.1-dev, SDXL, and Z-Image.
- Test them on seen prompts and unseen prompts.
- Run a final Phase 6 generalization benchmark using 10 new product/object prompts.
- Evaluate the generated images with automated metrics and with a qualitative vision-language judge.

The reason for this multi-stage evaluation is important: one metric alone cannot capture text alignment, visual quality, style realism, and physical plausibility at the same time.

2. Why metadata and normalization mattered before training

Even though this document starts from fine-tuning, the fine-tuning stage only makes sense because the earlier pipeline created consistent metadata. In this project, raw annotations contained technique names, weave types, finish information, post geometry, weft geometry, and free-text notes. Without normalization, the caption model would see mixed naming styles and inconsistent structures.

The normalized schema used top-level fields such as task_id, image, technique, weave_types, finish, posts, wefts, special_description, and bbox. That gave the caption stage one stable language before the training stage started.

This matters for training because prompt quality is one of the strongest hidden variables in LoRA success. Clean captions give cleaner supervision.

3. LoRA concept and where it was attached

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning method. Instead of updating every model weight, LoRA freezes the main pretrained weights and learns small low-rank update matrices. In practical project terms, this means: keep the base model intelligence, inject the weaving style into selected layers, and reduce compute cost.

The report explains LoRA as $W = W_0 + (\alpha/r) * B * A$. The important practical meaning is simpler:

- W_0 = the original pretrained weights stay frozen.
- A and B = small trainable matrices that learn the style update.
- rank controls how much flexibility the update has.
- α scales the update strength.

In this project, LoRA rank and α were both set to 32 for all three backbones. The exact target modules differed by model, which is where architecture and fine-tuning strategy meet.

4. Fine-tuning setup for each model

4.1 FLUX.1-dev fine-tuning setup

Parameter	Value
Base model	black-forest-labs/FLUX.1-dev
Text encoders	T5-XXL + CLIP ViT-L
Hardware	Multi-GPU with accelerate
LoRA rank / α	32 / 32
Target modules	Cross-attention and feed-forward projections in double-stream blocks
Learning rate	1e-4
Precision	bfloat16
dataset_repeat	20
Dataset size	177 training images
Completed training	Epoch 0 to Epoch 1 (400 cumulative steps)

Project reading: FLUX had access to the full 177-image dataset and a very strong text stack. That gave it the best chance to learn detailed prompt-to-visual mapping rather than only local texture imitation.

4.2 SDXL fine-tuning setup

Parameter	Value
Base model	stabilityai/stable-diffusion-xl-base-1.0
LoRA rank / α	32 / 32
Target modules	UNet cross-attention and projection weights
Hardware	Multi-GPU with accelerate
Learning rate	1e-4
Precision note	float32 VAE decoding required
dataset_repeat	20 (3540 instances per epoch)
Dataset size	177 training images
Completed training	Epochs 0, 1, 2, 3

Important issue from the repo: SDXL VAE decoding in float16 caused black images because of numerical underflow. So the project explicitly forced float32 decoding for the VAE. This is a strong example of architecture-specific engineering detail affecting practical training.

4.3 Z-Image fine-tuning setup

Parameter	Value
Base model	Z-Image (DiT architecture)
LoRA rank / alpha	32 / 32
Training dataset	10 images
Epoch 0 strategy	dataset_repeat = 50 to force style acquisition
Epochs 1-4 strategy	dataset_repeat = 20 for regularization
Total steps by Epoch 4	1300
Validation set	10 seen + 50 unseen prompts

This was the riskiest setup. Z-Image was trained on only 10 images, so its strongest advantage (high-detail structure specialization) also came with the highest risk of overfitting.

5. Internal architecture explanation of SDXL

SDXL is a latent diffusion model. It does not denoise full-resolution images directly. Instead, it compresses images into a latent space through a VAE, denoises that latent representation with a UNet backbone, and decodes it back into the output image.

A simple internal path looks like this:

```
Text prompt -> dual text encoders -> conditioning vectors -> noisy latent -> UNet denoiser -> clean latent -> VAE decoder -> final image
```

- Dual text encoders: CLIP ViT-L and OpenCLIP ViT-G.
- Latent space generation: cheaper than full pixel-space denoising.
- UNet backbone: downsampling path, bottleneck, upsampling path, skip connections, and attention blocks.
- Cross-attention: where prompt features control image regions.
- Extra conditioning: size and crop conditioning help control framing and resolution behavior.

Why it struggled in this project: the VAE bottleneck is efficient, but it can smooth away the finest weave details. That is why border stitching, micro-texture, and precise interlocking strand depth were weaker in SDXL outputs.

6. Internal architecture explanation of FLUX.1-dev

FLUX.1-dev replaces the UNet with a transformer-based denoiser and uses a rectified flow formulation. In practical terms, it treats image content more like a sequence of tokens or patches than a grid processed by convolutional down/up paths.

A simple internal path looks like this:

```
Prompt -> T5-XXL + CLIP -> text tokens; noisy image latent -> patch tokens -> double-stream transformer blocks -> single-stream transformer blocks -> denoised prediction
```

- Double-stream blocks: text tokens and image tokens are first processed in partly separate streams.
- Joint attention in the early stage lets language and image interact while preserving modality-specific structure.
- Single-stream blocks: later both token streams are fused into one unified transformer sequence.
- Rectified flow / flow matching: predicts a direction of movement from noise to data rather than classic direct noise prediction.
- This architecture scales well and handles long descriptive prompts strongly.

Why it worked well in this project: FLUX combined a large transformer backbone with a strong language stack, which made it especially effective for material attributes, gloss, border details, and prompt adherence.

7. Internal architecture explanation of Z-Image

Z-Image is treated in the project as a diffusion-transformer-style model specialized for structured, repeated high-frequency patterns. Public block-level documentation is less transparent than SDXL or FLUX, so part of the explanation in the project is functional and behavior-based rather than fully official reverse-engineering.

Project-side interpretation:

- Dense positional embeddings help the model care about exact spatial arrangement.
- Spatial attention helps preserve repeated lattices and aligned crossings.
- The model is especially suited to structured pattern realism rather than only general photo generation.
- Public Z-Image-Turbo notes additionally describe a single-stream S3-DiT style design for efficiency.

Why this matters for weaving: woven geometry is not just texture; it is repeated spatial logic. Z-Image appears to have a structural advantage there, which is consistent with its strong visual style realism in the project results.

8. Phase 6 generalization prompt set

Phase 6 tested whether the learned intrecciamei-style could transfer beyond texture blocks into new objects. The project introduced 10 new prompts, including a lamp shade, handbag, decorative basket, headboard, wall hanging, stool seat, tray, vase cover, coaster, and office chair.

These prompts were intentionally more difficult because they required both style transfer and object identity preservation. This matters: a model can look good on texture close-ups and still fail when asked to wrap the same weave logic around a new object.

Prompt group	Example challenge
Lamp shade	Need lattice texture plus shadow behavior and lighting
Handbag	Need tight leather weave with luxury product cues
Stool seat	Need diagonal woven structure that still looks buildable
Office chair	Need breathable woven mesh on a new ergonomic shape
Headboard / tray / vase cover	Need style transfer onto product geometry without melting the weave

9. Automated evaluation pipeline

The project used `phase6/evaluate_metrics.py` to compute CLIPScore, LPIPS, and CLIP-IQA on generated images stored in each model folder. The script took three key inputs: `image_dir`, `ref_dir`, and `device`.

- `image_dir` = generated .png files and their prompt sidecar .txt files
- `ref_dir` = real reference images used for LPIPS comparisons
- `device` = cuda for GPU execution

The script found all generated PNG images, opened the matching prompt text file, computed three metrics, then wrote both CSV and Markdown reports. This created an objective benchmark before any human-style judgment was added.

9.1 CLIPScore in the project

The script loaded CLIP ViT-B/32, encoded the image and the prompt, normalized both embeddings, then computed cosine similarity. This measured prompt-image semantic alignment. Higher was better.

9.2 CLIP-IQA in the project

The script used CLIP again, but this time compared the image against a positive quality prompt and a negative quality prompt. After a softmax, it kept the probability of the good-quality description. This produced a reference-free image quality score between 0 and 1. Higher was better.

9.3 LPIPS in the project

The script loaded LPIPS with an AlexNet feature extractor, loaded up to 20 reference images, resized references and generated images to 512 x 512, and compared each generated image against all references. It kept the minimum distance. Lower was better.

The resizing step was not optional polish; it was a real engineering fix. Without matching image size, LPIPS could fail from tensor dimension mismatch.

10. Range and interpretation of metric values

Metric	Range in practice in this project	Higher or lower?	Interpretation
CLIPScore	Positive values around 0 to 1	Higher	Prompt-image match
CLIP-IQA	0 to 1	Higher	Reference-free image quality

Metric	Range in practice in this project	Higher or lower?	Interpretation
LPIPS	0 and above	Lower	Distance to closest real reference image

Important clarification: these automated metrics are not the same as the 0 to 5 judge scores. The 0 to 5 range belongs to `mllm_judge.py`.

10.1 Automated metric results

Model	CLIPScore	LPIPS	CLIP-IQA	What that suggests
FLUX	0.3392	0.0015	0.4784	Best prompt adherence
SDXL	0.3058	0.0013	0.3937	More moderate, weaker image quality score
Z-Image	0.3124	0.0010	0.5952	Best visual quality and best style closeness

This result pattern is one of the central findings of the project: FLUX best matched the text, while Z-Image best matched visual style realism.

11. MLLM-as-a-judge pipeline and 0 to 5 scoring

mllm_judge.py is the qualitative evaluation stage. Instead of using embeddings or distances, it asks a vision-language model to inspect each image and score it like an expert reviewer. In the project, the script was set up to use a Qwen vision model through Ollama.

For each generated image, the script:

- finds the matching prompt .txt file
- sends image + prompt to the judge model
- asks for strict JSON output
- extracts five 0.0 to 5.0 scores plus a reasoning string
- computes a per-image mean and a folder-level mean
- writes qualitative_scoring_report.csv and mllm_judge_report.md

11.1 The five judge criteria

Criterion	Meaning in this project
Prompt Adherence	Does the image include the requested content?
Intreccio Identity	Does it really look like authentic woven leather/rattan?
Manufacturability	Could a craftsperson physically build these crossings?
Visual Quality	Sharpness, lighting, aesthetic clarity
Controlled Originality	Is the style applied cleanly to the intended object?

All five are judged on a 0.0 to 5.0 scale. This is the range visible in the qualitative reports and in mllm_judge.py.

11.2 Judge results

Model	Prompt	Identity	Manufacturability	Quality	Originality	Overall mean / 5
FLUX	4.50	4.00	4.20	4.50	4.00	4.24
SDXL	4.10	3.80	3.99	4.31	3.65	3.97
Z-Image	4.20	3.90	3.98	4.29	3.75	4.02

This shows that the judge favored FLUX overall, even though Z-Image led the automated visual metrics. That difference is meaningful: FLUX looked more semantically controlled to the judge, while Z-Image looked more visually strong in automated image-quality/style terms.

12. Final model-by-model interpretation

FLUX.1-dev

- Best CLIPScore in the automated metrics.
- Best overall MLLM judge score.
- Strongest for detailed prompt adherence and semantic control.
- Architecture explanation supports this: powerful transformer backbone plus T5-XXL text understanding.

SDXL

- Strong and stable base model, but weaker than FLUX and Z-Image for this high-frequency woven-detail task.

- Automated scores lower on CLIPScore and CLIP-IQA than the other two.
- Architecture explanation supports this: latent-diffusion efficiency is useful, but the VAE bottleneck can wash out micro-detail.

Z-Image

- Best CLIP-IQA and best LPIPS.
- Strong visual realism and strongest closeness to real woven reference style.
- More fragile fine-tuning setup because it used only 10 training images.
- Architecture explanation supports this: structural/spatial specialization helped style realism, but small data increased overfitting risk.

12.1 Final project takeaway

The project did not produce a single universal winner. Instead, it showed three different strengths: FLUX for text adherence and controlled generation, Z-Image for visual realism and style similarity, and SDXL as a more general but less specialized baseline for this domain. This is the most honest project-level conclusion, because it reflects both the architecture design and the evaluation evidence.

12.2 Short presentation version

If you need one short summary for a teammate or presentation: FLUX understood the prompts best, Z-Image looked the most visually realistic, and SDXL was solid but less suited to the smallest woven details in this setup.

Source basis for this PDF: final_report.md, phase6.md, summary_flux.md, summary_sdsl.md, summary_zimage.md, automated_metrics_report.md, and mllm_judge_report.md.